

VEHICLE PARKING MANAGEMENT SYSTEM

1. Introduction

The Vehicle Parking Management System is a Python-based project developed to manage vehicles in a parking area. The system helps maintain vehicle records, assign parking slots, calculate parking charges, and manage vehicle entry and exit.

The project provides a simple menu-driven system that reduces manual work and makes basic parking management easier.

2. Problem Statement

Managing vehicles manually in a parking area can be time-consuming and may lead to mistakes in maintaining records, identifying available slots, and calculating parking fees.

The Vehicle Parking Management System provides a simple computerized solution for maintaining parking information and performing basic parking operations.

3. Objectives

- * To maintain records of parked vehicles.
- * To assign available parking slots.
- * To display occupied and available slots.
- * To calculate parking charges.
- * To manage vehicle entry and exit.

- * To reduce manual record keeping.
- * To apply Python programming concepts to a real-world problem.

4. Scope

The system can be used for small and medium-sized parking areas such as:

- * College parking
- * Office parking
- * Apartment parking
- * Shopping areas
- * Private parking spaces

The system can later be expanded with databases, online booking, digital payments, and automatic vehicle identification.

5. Target Users

The system can be used by:

- * Parking attendants
- * Parking lot owners
- * College administration
- * Office administrators
- * Apartment management

6. Functional Requirements

The system should be able to:

1. Register a vehicle.
2. Store the vehicle number.
3. Store the vehicle type.
4. Assign an available parking slot.
5. Display parked vehicles.
6. Display available parking slots.
7. Process vehicle exit.
8. Calculate parking charges.
9. Display the parking receipt.
10. Display the total parking collection.

7. Non-Functional Requirements

- * The system should be simple and user-friendly.
- * The system should provide accurate calculations.
- * The program should respond quickly.
- * The system should be easy to maintain.
- * The program should handle basic invalid inputs.
- * The system should work reliably for small parking areas.

8. System Architecture

The system follows a simple ****Input → Process → Output**** structure.

Input

- * Vehicle number
- * Vehicle type
- * Parking duration
- * Menu choice

Processing

- * Check slot availability
- * Assign parking slot
- * Store vehicle information
- * Calculate parking fee
- * Remove vehicle after exit
- * Update available slots
- * Update total collection

Output

- * Parking confirmation
- * Parked vehicle details
- * Available slots
- * Parking receipt
- * Total parking collection

9. Workflow

1. Start the program.
2. Display the main menu.
3. Select an operation.
4. Enter vehicle details.
5. Check whether a parking slot is available.
6. Assign a slot to the vehicle.
7. Store the vehicle information.
8. Display the parking details.
9. When the vehicle exits, enter the vehicle number.
10. Enter the parking duration.
11. Calculate the parking fee.
12. Remove the vehicle from the parking record.
13. Update the available slot.
14. Display the parking receipt.
15. Continue or exit the program.

10. Parking Fee Calculation

The system uses different hourly rates for different vehicle types.

Vehicle Type	Rate per Hour
-----	-----:
Bike	₹20

| Car | ₹40 |

Formula

****Parking Fee = Parking Duration × Rate per Hour****

Example

If a car is parked for 3 hours:

****Parking Fee = 3 × ₹40****

****Parking Fee = ₹120****

If a bike is parked for 4 hours:

****Parking Fee = 4 × ₹20****

****Parking Fee = ₹80****

11. Implementation

The project is implemented using Python and uses basic programming concepts.

The main concepts used are:

- * Variables
- * Input and output
- * Conditional statements
- * Loops
- * Functions
- * Lists
- * Dictionaries

A dictionary is used to store information about parked vehicles, while functions are used to divide the program into different operations.

12. Main Modules

12.1 Vehicle Entry Module

This module accepts the vehicle number and vehicle type. It checks whether the vehicle is already parked and assigns an available slot.

12.2 Parking Management Module

This module stores information about vehicles currently parked and keeps track of their parking slots.

12.3 Available Slot Module

This module checks occupied slots and displays the slots